

TECHNICAL DOCUMENTATION



Stackly Route

AI-Powered Logistics Route Optimization — Front-End Theme. A complete, static implementation covering the marketing site, role-based authentication, and two operational dashboards.

PREPARED BY

Manjunath D

DOCUMENT VERSION

1.0

STACK

HTML · CSS · JS

Table of Contents

The document follows the structure of the repository itself, from product overview through deployment.

01 Introduction

Project overview · Purpose and objectives · Scope · Technology stack · Key capabilities

02 Project Structure

Directory layout · Stylesheet and script inventory · Asset organization

03 Design System & Color Palette

Brand surfaces · Accent and semantic colors · Text and borders · Typography · Spacing, radius and effects

04 Application Pages

Marketing pages · Authentication · Fleet Manager Dashboard · Driver Dashboard · Error page

05 Responsiveness

Approach · Breakpoints · Component behavior · Accessibility and reduced motion

06 Deployment — GitHub Pages

Prerequisites · Publishing steps · Custom domain · Operational notes

07 Conclusion

What was delivered · Strengths · Future scope

Introduction

Project Overview

Stackly Route is a complete front-end implementation of an AI-powered logistics route optimization platform. The project is a static, framework-free theme built with plain HTML, CSS, and JavaScript, and it ships as one deployable unit: a marketing site that tells the product story, an authentication flow with two role types, and two role-specific dashboard applications.

Everything in the repository is authored by hand. There is no build step, no package manager, and no framework to learn. The only runtime dependencies are the two font families (Inter and Space Grotesk) and the Chart.js charting library, both loaded from a CDN. As a result the theme can be opened directly in a browser from the local file system, and it can be pushed to any static host with zero configuration.

Purpose and Objectives

The theme was built to satisfy a focused set of product and engineering goals:

- Present the Stackly Route brand and product narrative through a cinematic, section-driven marketing site.
- Demonstrate role-based access by routing Fleet Managers and Drivers into separate applications after sign-in.
- Provide a realistic operations console backed by mock fleet, route, and driver data rendered entirely client-side.
- Remain fully responsive from small phones to large desktop displays without losing hierarchy or readability.
- Keep the code lightweight, accessible, and easy to hand over or extend.

Scope

Included. Eleven pages spanning marketing, authentication, error handling, and two dashboards; a reusable design system expressed as CSS custom properties; a documented responsive strategy; demo-level session handling using the browser's `localStorage`; and the full asset set — brand logos, product photography, and the hero background video.

Out of scope. There is no backend, no real routing engine or mapping service, no persistence beyond `localStorage`, and no production-grade authentication. All operational data — vehicles, routes, drivers, and delivery counts — is simulated demo content used to exercise the interface. Forms validate and simulate submission on the client only.

Technology Stack

LAYER	TECHNOLOGY	NOTES
-------	------------	-------

Markup	HTML5	Semantic structure, ARIA roles and labels, meta descriptions on every page.
Styles	CSS3	Custom properties, grid and flexbox layout, media queries, keyframe animations, glassmorphism effects.
Scripting	Vanilla JavaScript	IIFE modules scoped to each page; no framework or build tooling.
Typography	Inter · Space Grotesk	Served from Google Fonts with system-font fallbacks.
Charts	Chart.js 4.4.4	Loaded on the two dashboards only, from jsDelivr CDN.
Imagery	WebP · MP4	Optimized photography, poster fallback for the hero video.
Version control	Git	Repository tracks the full project history.

Key Capabilities

- AI route planning and optimization showcase with animated route visuals.
- Real-time fleet tracking imagery and live-status cards.
- Smart ETA prediction and dynamic re-routing narratives.
- Fuel and cost optimization feature set with progress figures.
- Performance analytics marketing page plus an in-console analytics view.
- Demo authentication with seeded users and role-based redirects.
- Fleet Manager and Driver single-page applications.
- Full responsiveness and reduced-motion support.

Project Structure

The repository root doubles as the site root. All asset references are relative, so the same folder that opens locally is exactly what gets served in production.

Directory Layout

Logistics Route Optimization Theme/

index.html

Home / landing page

404.html

Not-found page

css/

14 shared stylesheets

base · navbar · hero · sections · footer · animations · responsive · pages · solutions · route-optimization · analytics · about · contact · auth

js/

7 shared scripts

main · solutions · route-optimization · analytics · about · contact · auth

pages/

8 marketing & auth pages

solutions · route-optimization · analytics · about · contact · login · signup

dashboard/

Fleet Manager SPA

index.html · css/dashboard.css · js/dashboard.js

driver-dashboard/

Driver / field SPA

index.html · css/driver.css · js/driver.js

assets/

Static media

logo-stackly.webp · logo-icon.webp · video/hero-bg.mp4 · images/ (16 WebP)

Stylesheet Inventory

The stylesheets are split by responsibility rather than by size. Base and responsive act globally; every other file maps to the page or component family it styles. This makes a change traceable to a single file.

FILE	RESPONSIBILITY
base.css	Design tokens (CSS variables), reset, base typography, buttons, containers, utility and screen-reader helpers.
navbar.css	Fixed header, nav indicator, and the off-canvas mobile menu.
hero.css	Cinematic home hero: video background, animated route paths, trust row, scroll cue.
sections.css	Homepage building blocks — marquee, story comparison, showcase dashboard, features, process, stats, visibility, testimonial, insights, final CTA.
footer.css	Site footer, link columns, newsletter form, social links.

<code>animations.css</code>	Scroll reveals, marquee loop, route line drawing, count-up figures.
<code>responsive.css</code>	Every media-query override, consolidated in one file.
<code>pages.css</code>	Shared subpage hero, background grids, placeholder and content scaffolding.
<code>solutions.css</code>	Solutions page: platform overview, deep-dive cards, ecosystem grid.
<code>route-optimization.css</code>	Route Optimization page: engine narrative, before/after compare, map demo.
<code>analytics.css</code>	Analytics page: pipeline visuals, intelligence panels.
<code>about.css</code>	About page: journey, philosophy, team layout.
<code>contact.css</code>	Contact page: form panel, office details, network illustration.
<code>auth.css</code>	Login and signup split layouts, role selection cards, form fields.

Script Inventory

Scripts follow the same one-file-per-surface convention. Each is an IIFE that attaches to a specific page, keeps no global state beyond the demo session, and cleans up after itself.

FILE	RESPONSIBILITY
<code>main.js</code>	Home page: sticky navbar, mobile menu, scroll reveals, animated counters, marquee, newsletter handling.
<code>solutions.js</code>	Solutions page interactions and animated figures.
<code>route-optimization.js</code>	Route Optimization page: animated map demo, before/after figures, counters.
<code>analytics.js</code>	Analytics page: pipeline animation and performance figures.
<code>about.js</code>	About page: scroll-driven reveals and impact figures.
<code>contact.js</code>	Contact page: form validation, inline errors, network illustration animation.
<code>auth.js</code>	Login/signup: field and role validation, <code>stackly_users</code> seeding, session creation, role-based redirects.
<code>dashboard/js/dashboard.js</code>	Fleet Manager SPA: view router, mock fleet/route data, Chart.js rendering, session gating (~1,900 lines).
<code>driver-dashboard/js/driver.js</code>	Driver SPA: view router, route progression flow, Chart.js rendering, session gating (~1,200 lines).

Asset Organization

Media lives under a single `assets/` folder: brand files at the top, the hero video in `video/`, and product photography in `images/`. Every image is WebP with explicit dimensions so layout does not shift during load; the hero uses an MP4 with a poster fallback for slow connections.

Design System & Color Palette

All colors, type scales, radii, shadows, and transitions are declared once as CSS custom properties in `base.css` and consumed everywhere else. The system below is the complete token set.

Color Philosophy

The interface runs on a near-black, deep-navy canvas. Cyan is the single dominant accent — it doubles as the route color, communicates motion and technology, and is used sparingly for large surfaces. A small set of semantic colors (violet, emerald, amber, rose) is reserved for feature icons and status states, never for backgrounds or body copy. Text follows a three-step hierarchy of slate tones that keeps contrast comfortable against the dark surfaces.

Brand Surfaces

#030712 Canvas <code>--color-bg</code> Page background. The darkest tone, used for every marketing page body.	#0B1120 Elevated <code>--color-bg-elevated</code> Subpage hero backgrounds and raised sections that need to lift off the canvas.	#111827 Surface <code>--color-bg-card</code> Cards, panels, and dashboard surfaces that sit above the page background.
rgba(17, 24, 39, .72) Glass <code>--color-bg-glass</code> Translucent surfaces over media, paired with backdrop blur.	rgba(17, 24, 39, .42) Glass Light <code>--color-bg-glass-light</code> Lighter glass tint for secondary overlays and stat chips.	

Accent & Semantic Colors

#06B6D4 Accent <code>--color-accent</code> Primary brand cyan: buttons, links, active states, route paths, eyebrows.	#22D3EE Accent Light <code>--color-accent-light</code> Hover states and gradient ends for the accent color.	#0891B2 Accent Dark <code>--color-accent-dark</code> Pressed states and the lower bound of accent gradients.
---	--	---

#8B5CF6

Violet
`--color-violet`
Secondary accent for feature icons and route nodes. Glow: `rgba(139,92,246,.35)`.

#10B981

Emerald
`--color-emerald`
Positive states: savings, on-time delivery, success indicators.

#F59E0B

Amber
`--color-amber`
Warnings and attention states, fuel-efficiency visuals.

#F43F5E

Rose
`--color-rose`
Errors, risk states, and the access-denied indicators.

Text & Border Colors

#F8FAFC

Text Primary
`--color-text-primary`
Headings and primary copy on dark backgrounds.

#94A3B8

Text Secondary
`--color-text-secondary`
Body copy, descriptions, and supporting text.

#64748B

Text Muted
`--color-text-muted`
Captions, labels, and disabled content.

rgba(148,163,184,.12)

Border
`--color-border`
Default hairline dividers and card outlines.

rgba(148,163,184,.20)

Border Strong
`--color-border-strong`
Outlined buttons and emphasized dividers.

Typography

Space Grotesk handles headings with tight, slightly negative tracking; Inter carries the body at a comfortable 1.6 line-height. Every type size is fluid, expressed with `clamp()` so text scales smoothly between breakpoints instead of jumping at fixed widths.

TOKEN	FLUID VALUE	RANGE
<code>--text-xs</code>	<code>clamp(0.75rem, 0.7rem + 0.25vw, 0.875rem)</code>	12–14px
<code>--text-sm</code>	<code>clamp(0.875rem, 0.8rem + 0.35vw, 1rem)</code>	14–16px

<code>--text-base</code>	<code>clamp(1rem, 0.9rem + 0.5vw, 1.125rem)</code>	16–18px
<code>--text-lg</code>	<code>clamp(1.125rem, 1rem + 0.65vw, 1.375rem)</code>	18–22px
<code>--text-xl</code>	<code>clamp(1.5rem, 1.2rem + 1.5vw, 2.25rem)</code>	24–36px
<code>--text-2xl</code>	<code>clamp(2rem, 1.5rem + 2.5vw, 3.5rem)</code>	32–56px
<code>--text-3xl</code>	<code>clamp(2.75rem, 2rem + 3.75vw, 5.5rem)</code>	44–88px

Spacing, Radius, and Effects

TOKEN	VALUE	PURPOSE
<code>--container-max</code>	1280px	Maximum content width on large displays.
<code>--container-padding</code>	<code>clamp(1rem, 4vw, 3rem)</code>	Fluid side padding for the container.
<code>--section-padding-y</code>	<code>clamp(3.5rem, 5vw, 5.5rem)</code>	Vertical rhythm between homepage sections.
<code>--radius-sm/md/lg/xl</code>	8 / 12 / 18 / 28px	Corner radii from chips to feature cards.
<code>--shadow-sm/md/lg</code>	1px–24px blur	Layered elevation; <code>--shadow-glow</code> adds a cyan halo to primary actions.
<code>--transition-*</code>	150 / 250 / 400 / 500ms	Fast to bounce easing curves used consistently across hover and reveal states.

Usage note. The palette is intentionally restrictive: cyan communicates interactivity, slate communicates information, and the four semantic hues appear only where their meaning adds clarity. This restraint is what keeps the dense dashboard screens legible.

Application Pages

Eleven pages make up the product surface. Each description below covers what the page exists to do and how it is put together.

Marketing & Product Pages

Home — Landing Page

[index.html](#)

The product's front door. A cinematic hero with background video, animated route lines, and a trust row sets the story before the fold. Below it, the page moves from the logistics problem to the Stackly solution: an optimization showcase with a live-looking command dashboard, six core feature cards, a five-step process, and a stats band. Testimonials, an insights CTA, and a final signup prompt close the narrative.

Styles: hero, sections, navbar, footer, animations, responsive · Script: main.js

Solutions

[pages/solutions.html](#)

Presents the platform in terms of business outcomes rather than features. It opens with a page hero and then groups content into platform overview, deep dives, adaptability, live operations, business impact, and ecosystem. Solution-specific photography is used throughout, and sections reveal on scroll. The page reads as a capability catalog for prospective customers.

Styles: pages, solutions · Script: solutions.js

Route Optimization

[pages/route-optimization.html](#)

Explains the optimization engine in narrative and visual form. A before/after route comparison, an animated map demo, and a multistop hub graphic make the algorithm tangible. The page also covers scaling, dynamic intelligence such as re-routing and live ETA, and a closing call to action.

Styles: pages, route-optimization · Script: route-optimization.js

Analytics

[pages/analytics.html](#)

Positions the analytics layer as the decision engine of the platform. It covers performance intelligence across fleet, warehouse, and control; walks the analytics pipeline from data ingestion to insight; and previews predictive intelligence. Rich imagery and animated panels support the narrative. This marketing page is distinct from the Analytics view inside the Fleet Manager dashboard.

Styles: pages, analytics · Script: analytics.js

About

[pages/about.html](#)

Tells the company story: mission, journey, philosophy, impact, and team. The layout is editorial, paced with imagery and animated reveals rather than dense panels. Copy stays product-agnostic and focused on the problem being solved, which gives the page a human, trustworthy tone.

Styles: pages, about · Script: about.js

Contact

[pages/contact.html](#)

A lead-generation page built around a validated contact form with office details and a closing CTA. The form validates name, email, and message, shows inline errors, and submits with a client-side handler. A network illustration sits behind the form to keep the page on-brand.

Styles: pages, contact · Script: contact.js

Authentication

Log In

[pages/login.html](#)

Split-screen authentication layout: a branded visual panel beside the sign-in form. The form collects email, role, and password, validates each field inline, and on success stores a session in `localStorage`. It then redirects by role — Fleet Managers to the management dashboard, Drivers to the driver app. Demo accounts are seeded on first run.

Styles: pages, auth · Script: auth.js

Sign Up

[pages/signup.html](#)

Companion to the login page, sharing the same split layout and stylesheet for a consistent flow. It collects name, email, role, password, and terms acceptance; validates everything inline; and appends the account to the local user store before routing back to login. The same role cards used on login keep the two forms visually paired.

Styles: pages, auth · Script: auth.js

Dashboards

Fleet Manager Dashboard

[dashboard/index.html](#)

A single-page application for operations managers. A persistent sidebar navigates six views rendered client-side: Overview, Fleet, Routes, Drivers, Analytics, and Settings. Fleet data for ten vehicles and eight routes ships as JavaScript mock objects, and performance charts are drawn with Chart.js. Access is gated by the demo session, showing an Access Denied panel when no session exists.

Styles: dashboard.css · Script: dashboard.js · Chart.js 4.4.4

Driver Dashboard

[driver-dashboard/index.html](#)

A mobile-first companion application for field operators. Six views cover Overview, My Route, Deliveries, Schedule, Performance, and Settings. The interface favors phones — generous touch targets, compact cards, and a collapsible sidebar — while staying usable on desktop. Drivers can walk a route through start and complete states; charts are rendered with Chart.js.

Styles: driver.css · Script: driver.js · Chart.js 4.4.4

Error Handling

404 Not Found

[404.html](#)

A branded error page that keeps users oriented. It explains the missing page in product voice and offers a way back to the home page and the main site sections. Styled consistently with the marketing site, so a dead end never feels like a broken experience.

Styles: base, navbar, footer

Cross-linking. Navigation and footer repeat across every marketing page, so users always have a path back to the primary surfaces. The dashboards intentionally drop the marketing footer in favor of their own app chrome.

Responsiveness

The theme is fluid first. Type scales with `clamp()`, containers and grids reflow rather than pinning pixel widths, and navigation swaps from an inline menu to a hamburger drawer at tablet width. Breakpoints tune layouts; they do not force them.

Breakpoints

All media queries are consolidated in `css/responsive.css` plus a global reduced-motion block in `base.css`. The breakpoints are:

BREAKPOINT	TARGET	WHAT CHANGES
<code>min-width: 1440px</code>	Large desktop	Container side padding grows to 4rem, keeping line lengths comfortable on very wide displays.
<code>max-width: 1024px</code>	Tablet	Desktop nav is hidden and the hamburger drawer appears. Features grid drops to two columns; story, visibility, and footer columns reflow; stats band becomes two columns.
<code>max-width: 900px</code>	Small tablet / portrait	Footer column gutters are tightened for narrower portrait widths.
<code>max-width: 640px</code>	Mobile	Single-column grids everywhere. Hero actions, insights, and CTA buttons stack full-width; testimonial metrics and stat cards stack; footer becomes one centered column.
<code>max-width: 420px</code>	Small phones	Footer spacing is reduced and newsletter input centers for very narrow screens.
<code>max-height: 700px</code>	Short screens	The hero collapses to a compact height and the scroll cue is hidden so the fold still fits on laptops with shallow viewports.

Component Behavior

Navigation

On desktop the primary nav is a centered inline menu with an animated indicator under the active link. At 1024px and below, the menu collapses into an off-canvas drawer containing the logo, links, and authentication buttons. The drawer locks body scroll while open, closes on Escape or backdrop tap, and carries proper `aria-expanded` state.

Dashboards

Both dashboards use a persistent left sidebar on wide screens. On mobile the sidebar becomes an off-canvas drawer toggled by a hamburger in the top bar; content reflows to a single column, cards

stack, and Chart.js canvases scale to their container. The driver app is tuned phone-first: its route and delivery cards, action buttons, and tab bars are sized for touch use.

Forms and media

Authentication and contact forms stack to a single column with full-width inputs and larger touch targets on small screens. Images are fluid (`max-width: 100%`), and the hero uses a video with a poster image fallback for slow connections and reduced-motion contexts.

Reduced Motion & Accessibility

Accessibility is treated as part of the responsive work, not an afterthought:

- A `prefers-reduced-motion` media query disables transitions and animations globally and keeps `scroll-behavior` instant.
- A visible skip link jumps straight to main content; focus states use a cyan outline via `:focus-visible`.
- Navigation, menus, and role cards expose ARIA roles, labels, and keyboard operability; form errors are announced with `aria-live` regions.
- The auth role cards are full `role="radio"` groups, selectable by keyboard with arrow-key navigation.

Verification. The theme was exercised at the five width breakpoints above, plus portrait phone height. No horizontal scroll is introduced at any width; grids, buttons, and hero elements all reflow cleanly.

Deployment — GitHub Pages

Because the theme is a static site with no build step, publishing it is a matter of serving the repository folder. GitHub Pages is the simplest fit: the project is already version-controlled with Git, and Pages serves the repo root directly with no configuration files.

Prerequisites

- A GitHub account and a repository for the project (the existing repository can be used as-is).
- Git installed locally, with the project committed on the `main` branch.
- All asset references already relative — no path rewrites are needed for subfolder or root hosting.

Publishing Steps

- 1 Push the project to GitHub. From the project folder, initialize if needed and push the working tree: `git init`, `git add .`, `git commit -m "Initial commit"`, `git branch -M main`, then `git remote add origin <repository-url>` and `git push -u origin main`.
- 2 In the repository on GitHub, open **Settings** → **Pages**.
- 3 Under **Build and deployment**, set the source to **Deploy from a branch**.
- 4 Select the `main` branch and the **/ (root)** directory as the source folder, then save. The home page is `index.html` at the root, so no folder needs to be specified.
- 5 GitHub builds and serves the site within about a minute. The live URL is `https://<username>.github.io/<repository>/`. The marketing pages under `pages/` and both dashboards resolve through their relative links automatically.

Custom Domain

Optional. Add a `CNAME` record that points your domain to `<username>.github.io` (or apex records, depending on the provider), then enter the domain under **Settings** → **Pages** → **Custom domain** and save. HTTPS is provisioned automatically by GitHub after DNS settles.

Operational Notes

- The demo session and seeded users live in the visitor's `localStorage`. Clearing browser storage resets the demo; each new visitor starts with the same seeded accounts.
- The contact form and newsletter are client-side demonstrations. There is no server endpoint, so submissions are validated and acknowledged in the browser only.
- Chart.js and the two font families load from CDNs. The site still renders without them — charts show their fallback markup and fonts fall back to system stacks.

Conclusion

The repository is deliberately small — a handful of HTML files, fourteen stylesheets, and nine scripts — yet it covers a full product surface: marketing, authentication, and two operational consoles.

What Was Delivered

The project pairs a polished, story-driven marketing site with two functional dashboard applications, all sharing one design system defined in CSS custom properties. The dashboards demonstrate real interface patterns: client-side view routing, mock data structures, chart rendering, role-based access gating, and a mobile-first driver workflow. The auth flow shows how a static site can still present a convincing sign-in experience by leaning on the browser's local storage.

Strengths

- Framework-free and dependency-light; runs anywhere, deploys instantly.
- One source of truth for design tokens across every page.
- Fully responsive with a documented, consolidated breakpoint strategy.
- Consistent accessibility, keyboard support, and reduced-motion handling.
- Clean file organization where each stylesheet and script maps to one surface.
- All media is optimized (WebP images, poster-backed video).
- Realistic demo data gives the dashboards a production feel.
- Semantic HTML and meta descriptions across every page.

Future Scope

Natural next steps are wiring the interface to a real backend: a routing engine and mapping provider to replace the simulated data, a REST or GraphQL API for vehicles and routes, WebSockets for live tracking, and server-side authentication. On the front-end, the same design system could be ported to a component framework, and the dashboards extended with export, print, and real-time notification views.

The theme delivers the complete Stackly Route experience — from the first marketing impression to the daily work of a dispatcher or driver — in a single, portable, hand-authored codebase.